



CEBU INSTITUTE OF TECHNOLOGY
U N I V E R S I T Y

IT342-[Section] System Integration and Architecture

System Design Document (SDD)

Project Title: PayWEAC

Prepared By: Carin, Wilfred John R.

Version: 1.0

Date: 02/03/2026

Status: Draft

REVISION HISTORY TABLE

Versio n	Date	Author	Changes Made	Status
0.1	02/02/2026	Wilfred John Carin	Initial draft	Draft
0.2	[Date]	Wilfred John Carin	Added API specifications	Review
0.3	[Date]	Wilfred John Carin	Updated database design	Review
0.4	[Date]	Wilfred John Carin	Added UI/UX designs	Review
0.5	[Date]	Wilfred John Carin	Incorporated feedback	Revised
0.6	[Date]	Wilfred John Carin	Final review and corrections	Final
1	[Date]	Wilfred John Carin	Baseline version for development	Approved

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY	4
1.0 INTRODUCTION	5
2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION	5
3.0 NON-FUNCTIONAL REQUIREMENTS	8
4.0 SYSTEM ARCHITECTURE	9
5.0 API CONTRACT & COMMUNICATION	10
Authentication Endpoints	11
User Registration	11
User Login	11
6.0 DATABASE DESIGN	13
7.0 UI/UX DESIGN	14
8.0 PLAN	17

EXECUTIVE SUMMARY

1.1 Project Overview

PayWEAC is a simplified payment and tracking platform that enables users to track their month's rent due or specific month and can use the platform to pay through online transactions.

The system consists of:

- Spring Boot backend API
- React web application
- Android mobile application

1.2 Objectives

1. Develop a functional app with authentication, payment method, and tracking progress
2. Implement a three-tier architecture using Spring Boot (backend), React (web), and Android (mobile)
3. Create APIs for communication between all system components
4. Design a simple and somewhat responsive interface on mobile platform
5. Deploy all system components to production-ready environments

1.3 Scope

Included Features:

- User registration and authentication (email/password)
- Tenant login/logout
- Admin login/logout
- Rent record management
- Online payment submission (QR-based)

- Payment approval workflow (Admin)
- Rent tracking (current, past, upcoming)
- Responsive web interface
- Android mobile application

Excluded Features:

- Email notification system
- SMS alerts
- Advanced financial analytics
- Multi-language support
- Automatic payment gateway integration
- Partial payment support (optional if not implemented)

1.0 INTRODUCTION

1.1 Purpose

This document serves as the comprehensive design specification for the PayWEAC system. It provides services to tenants or boarders to make payment easier.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: PayWEAC

Domain: Track/Online Payment

Primary Users: Tenants/Boarders and Landlords/Landladys

Problem Statement: Users need a simple and easy way to pay and track your rent dues or upcoming dues

Solution: A simple mobile app to focus on simple, fast, and easy functionality

2.2 Core User Journeys

Journey 1: First-time User

1. User downloads the app
2. Clicks "Sign Up" and creates account
3. Set up details(emails,room number/name)
4. Finish details

Journey 2: Payment Method

1. Click "Payment" or "Pay Now"
2. Choose payment option(Gcash Qr Code, cash)
3. Online payment method through QR code
4. Wait for confirmation of payment
5. Admin/Landlords confirms payment and approve the payment
6. If cash, wait till admin approves your payment and confirm your payment

Journey 3: Administrator Product Management

1. Admin logs in with admin credentials
2. Navigates to product management section
3. Approves payments and dues
4. Edits existing dues
5. Views list of all missed dues or unpaid dues
6. Manages every payment

2.3 Feature List (MoSCoW)

MUST HAVE

1. User authentication (register, login, logout)
2. Rent catalog (due rent, past rents, upcoming rents)
3. Online Payment (QR code)
4. Admin panel

SHOULD HAVE

1. Product categories and filtering

2. Order history for users
3. Basic input validation feedback
4. Responsive design for all screen sizes
5. Email notifications

COULD HAVE

1. Online Payment through bank transfer
2. Notifications about the rent due

WON'T HAVE

1. Advanced analytics
2. Multi-language support

2.4 Detailed Feature Specifications

Feature: User Authentication

1. **Screens:** Registration, Login, Forgot Password
2. **Fields:** Email, Password, Confirm Password, First name, Last name, Room Number
3. **Validation:** Email format, password strength, uniqueness
4. **API Endpoints:** POST /auth/register, POST /auth/login, POST /auth/logout
5. **Security:** JWT tokens, password hashing with bcrypt

Feature: Rent Management(Admin)

- **Screens:** Rent Records Page, Create Rent Form
- **Functions:** Create monthly rent for tenant, Edit rent details (amount, due date), View rent status, Mark overdue automatically (based on due date)
- **API Endpoints:** GET /admin/rents, POST /admin/rent, PUT /admin/rents/{id}, GET /admin/rents/overdue

Feature: Rent Tracking(Tenant)

- **Screens:** Dashboard, Rent History
- **Display Sections:**Current Month Rent, Past Unpaid Rent, Upcoming Rent
- **API Endpoints:** GET /rents/my-dues, GET /rents/history

Feature: Payment Submission

- **Screens:** Payment Page, Upload Proof Screen
- **Fields:** Payment Method (GCASH_QR / CASH), Reference Number, Proof Image Upload
- **Process:** Submit payment, Status = PENDING_APPROVAL, Admin reviews
- **API Endpoints:** POST /payments, GET /payments/my-payments

Feature: Admin Payment Approval

- **Screens:** Payments Page, Payment Review Modal
- **Functions:** View payment details, Approve payment, Reject payment (with reason)
- **API Endpoints:** GET /admin/payments, POST /admin/payments/{id}/approve, POST /admin/payments/{id}/reject

2.5 Acceptance Criteria

AC-1: Successful User Registration

- Given I am a new tenant
- When I enter valid email and password
- And confirm password matches
- And submit the registration form
- Then my account should be created
- And I should be logged in automatically
- And redirected to the dashboard

AC-2: Successful Rent Payment Submission

- Given I am logged in
- When I select a rent due
- And submit payment with valid reference and proof
- Then the payment status should be **Pending Approval**
- And it should appear in the admin payment list

AC-3: Admin Payment Approval

- Given I am logged in as Admin
- When I approve a pending payment
- Then the rent status should change to **Paid**
- And the payment status should become **Approved**

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API response time: ≤ 2 seconds for 95% of requests
- Web page load time: ≤ 3 seconds on broadband
- Mobile app cold start: ≤ 3 seconds
- Support 100 concurrent users
- Database queries complete within 500ms

3.2 Security Requirements

- HTTPS for all communications
- JWT token authentication
- Password hashing with bcrypt (salt rounds = 12)
- SQL injection prevention
- XSS protection
- Rate limiting: 100 requests/minute per IP
- Admin endpoints require role verification

3.3 Compatibility Requirements

- **Web Browsers:** Chrome, Firefox, Safari, Edge (latest 2 versions)
- **Android:** API Level 24+ (Android 7.0+)
- **Screen Sizes:** Mobile (360px+), Tablet (768px+), Desktop (1024px+)
- **Operating Systems:** Windows 10+, macOS 10.15+, Linux Ubuntu 20.04+

3.4 Usability Requirements

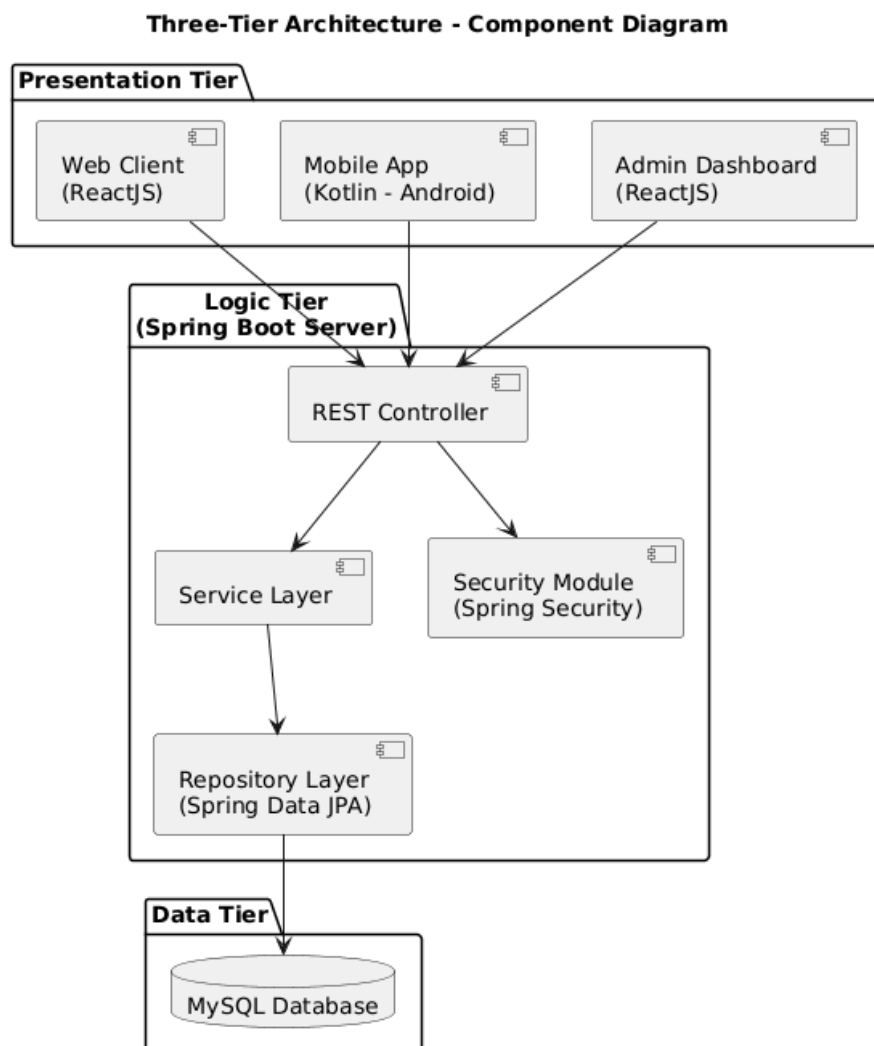
- Complete first purchase within 5 minutes for new users
- WCAG 2.1 Level AA compliance for web

- Consistent navigation across all pages
- Clear error messages with recovery options
- Touch targets minimum 44x44px on mobile
- Keyboard navigation support

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram

Note: This should be a component diagram



Technology Stack:

- **Backend:** Java17, Spring boot 3.x, Spring security, Spring data JPA
- **Database:** SQL
- **Web Frontend:** ReactJs
- **Mobile:** Kotlin, Android Studio
- **Build Tools:** Maven, Grandle
- **Deployment:** Backend – Railway / Render, Web – Vercel, Mobile – APK Distribution

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL	https://[server_hostname]:[port]/api/v1
Format	JSON for all requests/responses
Authentication	Bearer token (JWT) in Authorization header
Response Structure	{ "success": boolean, "data": object null, "error": { "code": string, "message": string, "details": object null }, "timestamp": string }

5.2 Endpoint Specifications

Authentication Endpoints

User Registration

Description	Register new tenant account
API URL	/auth/register
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	{ "email": "<email>", "password": "<password>", "confirmPassword": "<confirmPassword>", "firstName": "<firstName>", "lastName": "<lastName>", "roomNumber": "<roomNumber>"

	}
Response Structure	<pre>{ "success": true, "data": { "user": { "id": <id>, "email": "<email>", "firstName": "<firstName>", "lastName": "<lastName>", "role": "TENANT" }, "accessToken": "<token>", "refreshToken": "<token>" }, "error": null, "timestamp": "2026-02-21T10:30:00Z" }</pre>

User Login

Description	Authenticate tenant or admin
API URL	/auth/login
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	<pre>{ "email": "<email>", "password": "<password>" }</pre>
Response Structure	<pre>{ "success": true, "data": { "user": { "id": <id>, "email": "<email>", "firstName": "<firstName>", "lastName": "<lastName>", "role": "<TENANT ADMIN>" }, "accessToken": "<token>", "refreshToken": "<token>" }, "error": null, "timestamp": "2026-02-21T10:30:00Z" }</pre>

View My Rent Dues (Tenant)

Description	Retrieve tenant’s current, past, and upcoming rent dues
API URL	/rents/my-dues
HTTP Method	GET
Format	JSON for all requests/responses
Authentication	Bearer Token Required
Response Structure	{ "success": true, "data": { "currentMonth": { "month": "February 2026", "amount": 3500, "status": "UNPAID", "dueDate": "2026-02-28" }, "pastUnpaid": [{ "month": "January 2026", "amount": 3500, "status": "OVERDUE" }], "upcoming": [{ "month": "March 2026", "amount": 3500, "status": "PENDING" }] }, "error": null, "timestamp": "2026-02-21T10:30:00Z" }

Submit Payment(Tenant)

Description	Submit rent payment (GCash QR or Cash)
API URL	/payments
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	Bearer Token Required
Request Payload	<pre>{ "rentId": <rentId>, "paymentMethod": "GCASH_QR", "referenceNumber": "<referenceNumber>", "proofImageUrl": "<imageURL>" }</pre>
Response Structure	<pre>{ "success": true, "data": { "paymentId": <id>, "status": "PENDING_APPROVAL" }, "error": null, "timestamp": "2026-02-21T10:30:00Z" }</pre>

Admin Endpoints

Description	Create rent record for a tenant
API URL	/admin/rents
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	Bearer Token (Admin Role Required)
Request Payload	<pre>{ "tenantId": <tenantId>, "month": "March 2026", "amount": 3500, "dueDate": "2026-03-30" }</pre>

Response Structure	<pre>{ "success": true, "data": { "rentId": <id>, "status": "CREATED" }, "error": null, "timestamp": "2026-02-21T10:30:00Z" }</pre>
---------------------------	---

Approve Payment(admin)

Description	Approve tenant payment
API URL	/admin/payments/{id}/approve
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	Bearer Token (Admin Role Required)
Response Structure	<pre>{ "success": true, "data": { "status": "APPROVED" }, "error": null, "timestamp": "2026-02-21T10:30:00Z" }</pre>

5.3 Error Handling

HTTP Status Codes

- 200 OK – Successful request
- 201 Created – Resource created
- 400 Bad Request – Invalid input
- 401 Unauthorized – Authentication required/failed
- 403 Forbidden – Insufficient permissions
- 404 Not Found – Resource does not exist

- 409 Conflict – Duplicate resource
- 500 Internal Server Error – Server error

Error Code Examples

Example 1	<pre> json { "success": false, "data": null, "error": { "code": "AUTH-001", "message": "Invalid credentials", "details": "Email or password is incorrect" }, "timestamp": "2024-01-28T10:30:00Z" } </pre>
Example 2	<pre> { "success": false, "data": null, "error": { "code": "VALID-001", "message": "Validation failed", "details": { "email": "Email is required", "password": "Must be at least 8 characters" } }, "timestamp": "2024-01-28T10:30:00Z" } </pre>

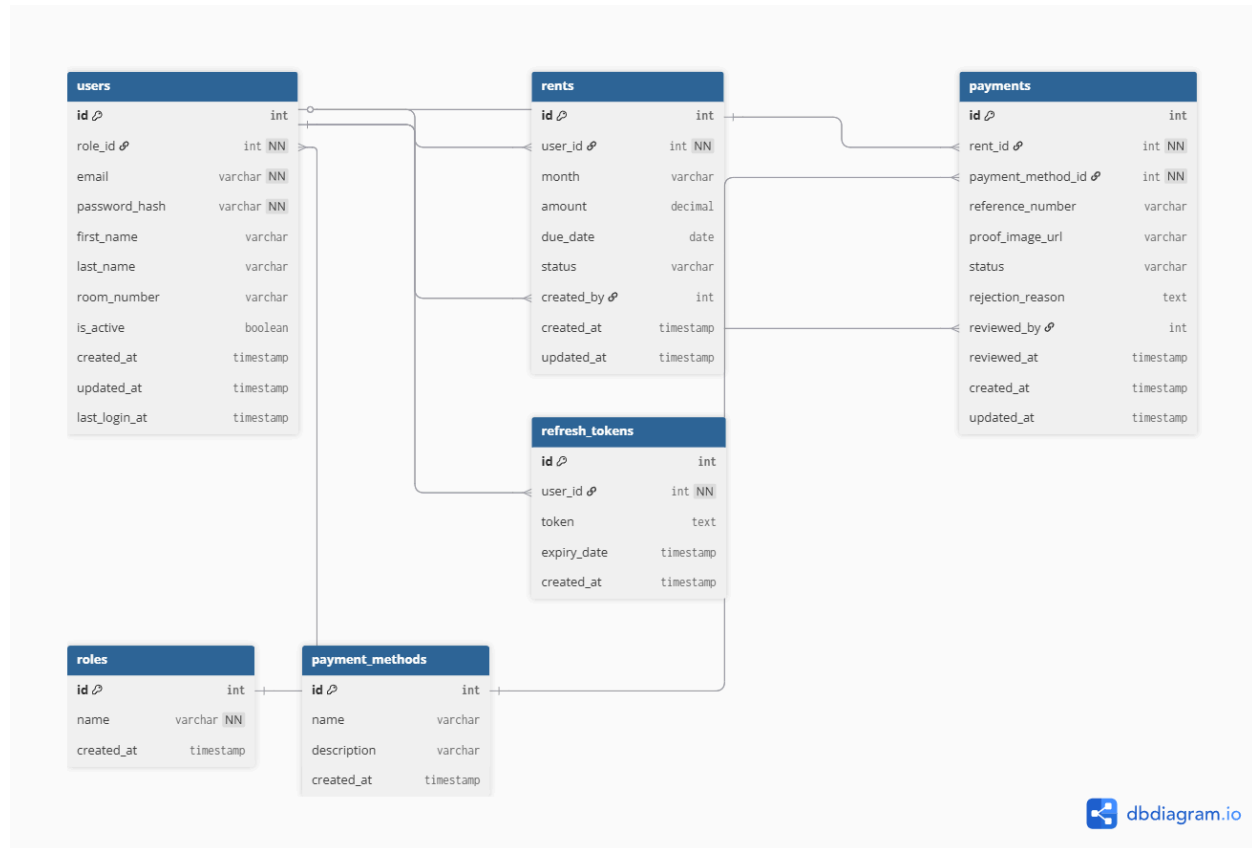
Common Error Codes

- AUTH-001: Invalid credentials
- AUTH-002: Token expired
- AUTH-003: Insufficient permissions
- VALID-001: Validation failed
- DB-001: Resource not found
- DB-002: Duplicate entry
- SYSTEM-001: Internal server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram

Note: This should be an ERD (Database Schema)



Detailed Relationships:

- **One User** → Many Rent Records
- **One Rent** → One Payment (if paid)
- **One Admin** → Approves Many Payments

Key Tables:

user

1. **id (PK)**
2. **email (unique)**

3. **password_hash**
4. **first_name**
5. **last_name**
6. **room_number**
7. **role (TENANT / ADMIN)**
8. **Created_at**

rents

- **id (PK)**
- **user_id (FK → users.id)**
- **month**
- **amount**
- **due_date**
- **status (PENDING, UNPAID, PAID, OVERDUE)**
- **created_at**

payments

- **id (PK)**
- **rent_id (FK → rents.id)**
- **payment_method (GCASH_QR, CASH)**
- **reference_number**
- **proof_image_url**
- **status (PENDING_APPROVAL, APPROVED, REJECTED)**
- **reviewed_by (FK → users.id)**
- **reviewed_at**
- **created_at**

refresh_tokens

- **id**
- **user_id**
- **token**
- **expiry_date**

Table Structure Summary:

- **users:** id, email, password_hash, full_name, role, created_at

7.0 UI/UX DESIGN

7.1 Web Application Wireframes

Note: This should be wireframes from Figma



Create Account

Register as a new tenant

First Name

John

Last Name

Doe

Email

john.doe@example.com

Room Number

101

Password

.....

Confirm Password

.....

Create Account

Already have an account? [Sign in](#)

Dashboard

Current Month Rent

Unpaid

February 2026

Amount Due

\$ ₱3,500

Pay Now

 Due: 2/28/2026

Past Due Rent

January 2026

Overdue

₱3,500

Due: 1/30/2026

Pay Now

Upcoming Rent

March 2026

Upcoming

₱3,500

Due: 3/30/2026

April 2026

Upcoming

Due: 4/30/2026

 Account created successfully!

Payment

[← Back to Dashboard](#)

Rent Details

Review your rent information

Month: February 2026

Due Date: 2/28/2026

Total Amount: ₱3,500

Payment Method

Choose how you want to pay

☒  **GCash QR Payment** Scan QR code and upload proof of payment

☐  **Cash Payment** Pay directly to the landlord

Scan this QR code with your GCash app



☒  **GCash QR Payment** Scan QR code and upload proof of payment

☐  **Cash Payment** Pay directly to the landlord

Scan this QR code with your GCash app



GCash QR Code

Reference Number *

Enter GCash reference number

Upload Proof of Payment *



Click to upload screenshot

Cancel

Submit Payment

Payment Confirmation



Payment Submitted Successfully!

Your payment is being reviewed by the administrator

⌚ Pending Approval

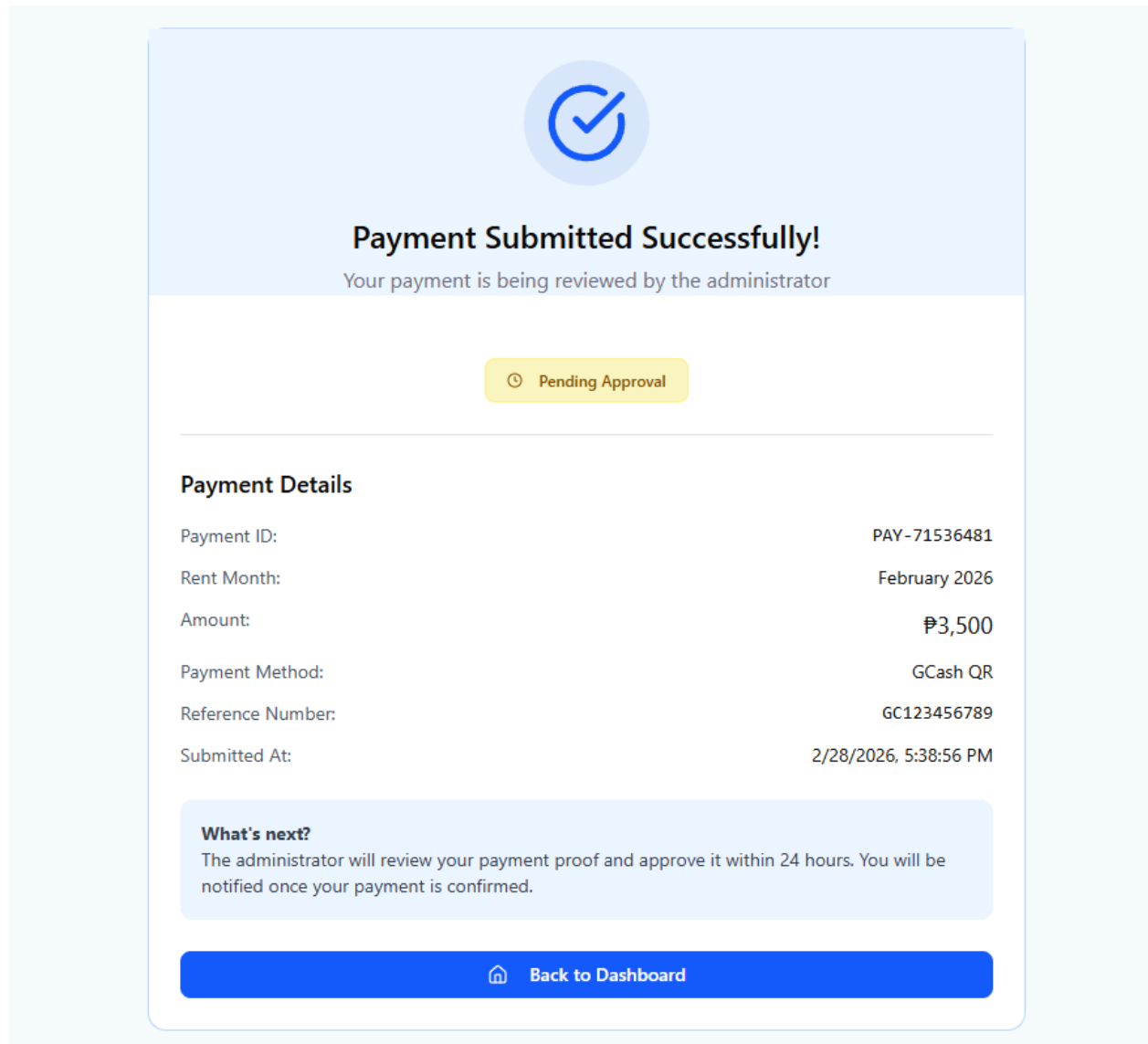
Payment Details

Payment ID:	PAY-71536481
Rent Month:	February 2026
Amount:	₱3,500
Payment Method:	GCash QR
Reference Number:	GC123456789
Submitted At:	2/28/2026, 5:38:56 PM

What's next?

The administrator will review your payment proof and approve it within 24 hours. You will be notified once your payment is confirmed.

[Back to Dashboard](#)



Main Content:

- Current Month Rent Card
- Past Due Rent List
- Upcoming Rent List
- "Pay Now" Button

Payment Page

- Display Rent Details

- Choose Payment Method:
 - GCash QR (show QR image)
 - Cash
- Upload Proof of Payment
- Submit Button

Payment Confirmation Page

- Status: Pending Approval
- Payment Reference
- Admin Review Status

Admin Dashboard

Sidebar:

- Dashboard
- Tenants
- Rent Records
- Payments
- Logout

Admin Features:

- Create Monthly Rent
- View All Tenants
- View Pending Payments
- Approve/Reject Payments
- View Missed/Overdue Rents

7.2 Mobile Application Wireframes

Note: This should be wireframes from Figma



PayWEAC

Sign in to your account to continue

Email

Password

[Forgot password?](#)

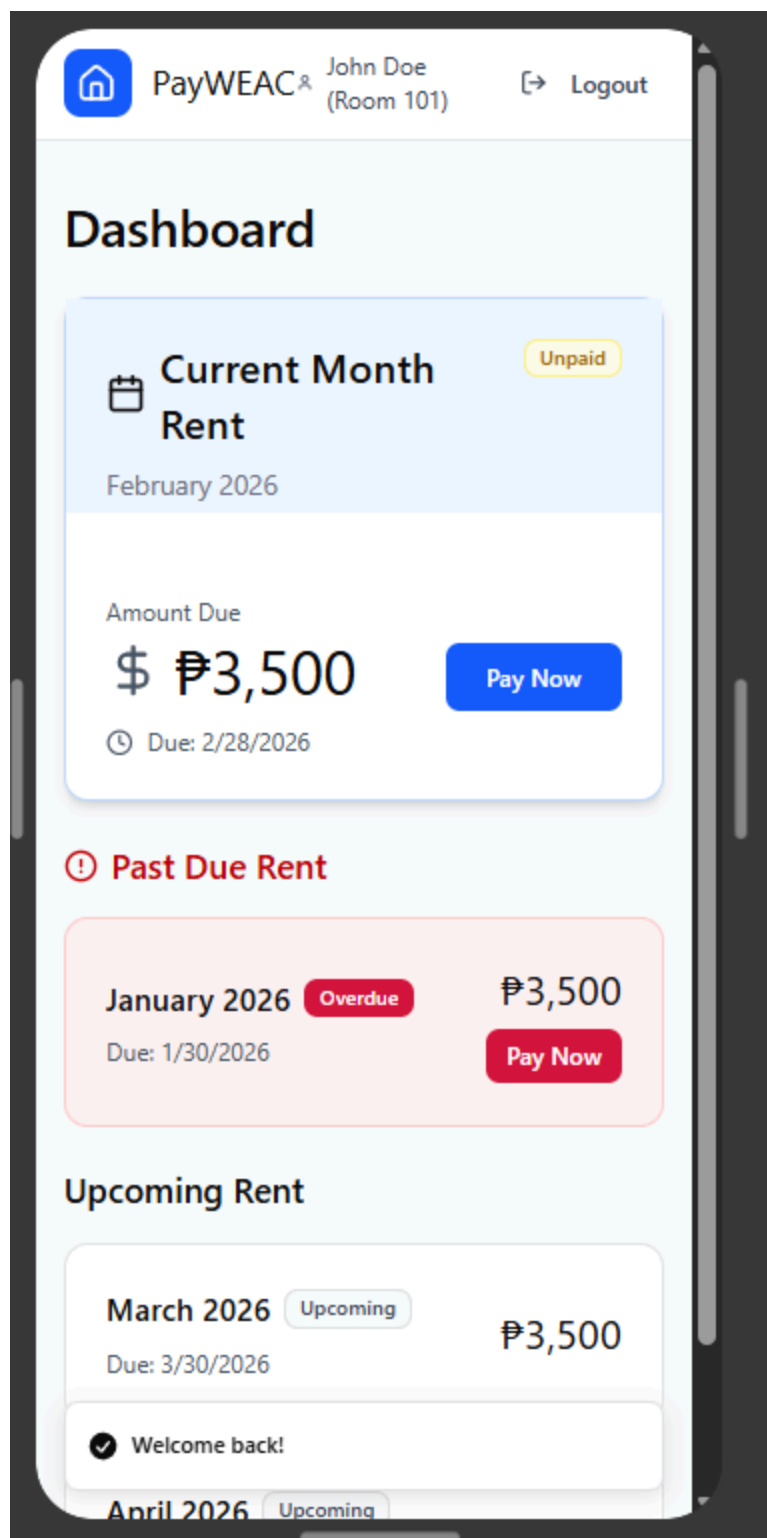
Sign In

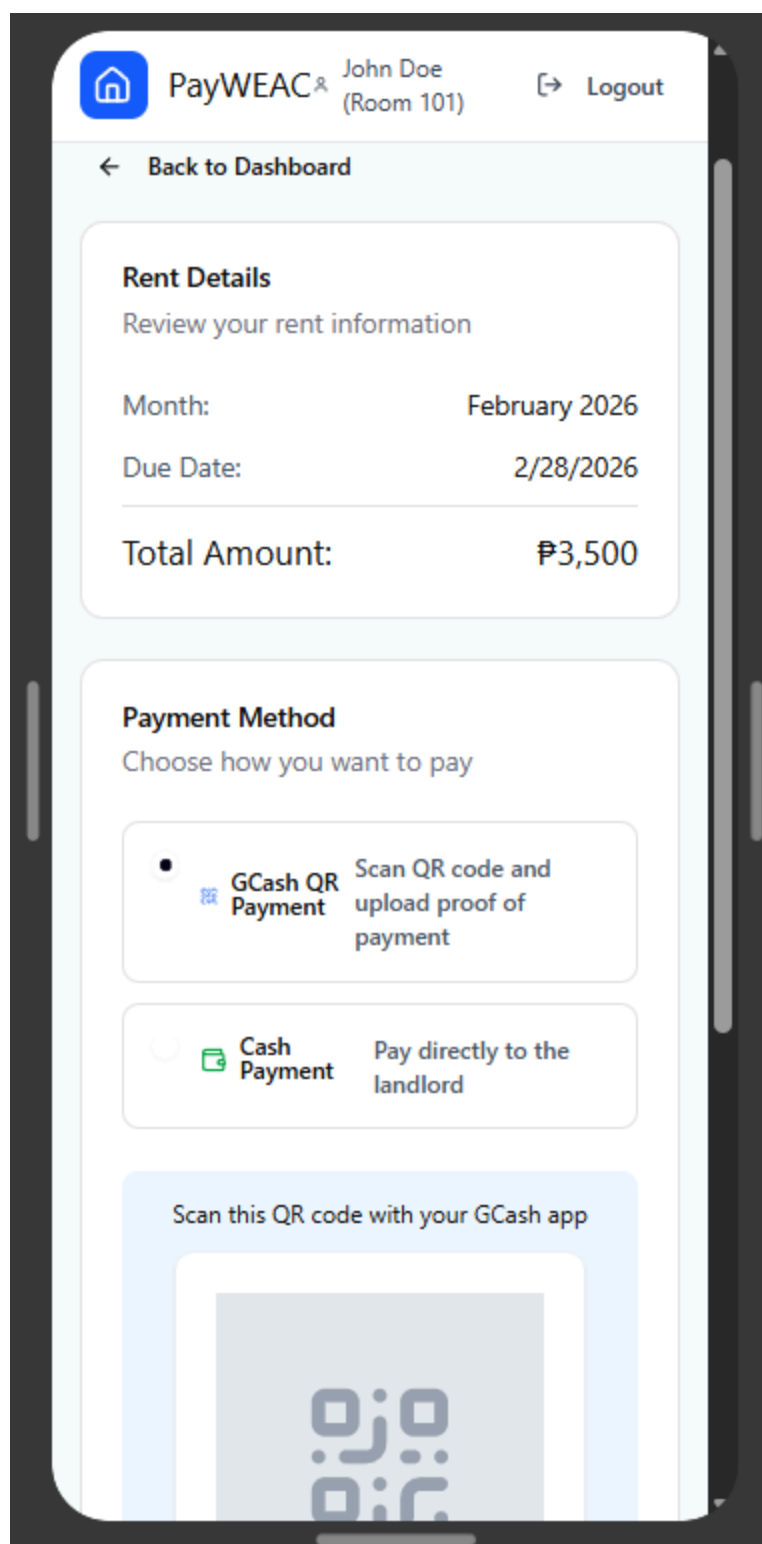
Don't have an account? [Sign up](#)

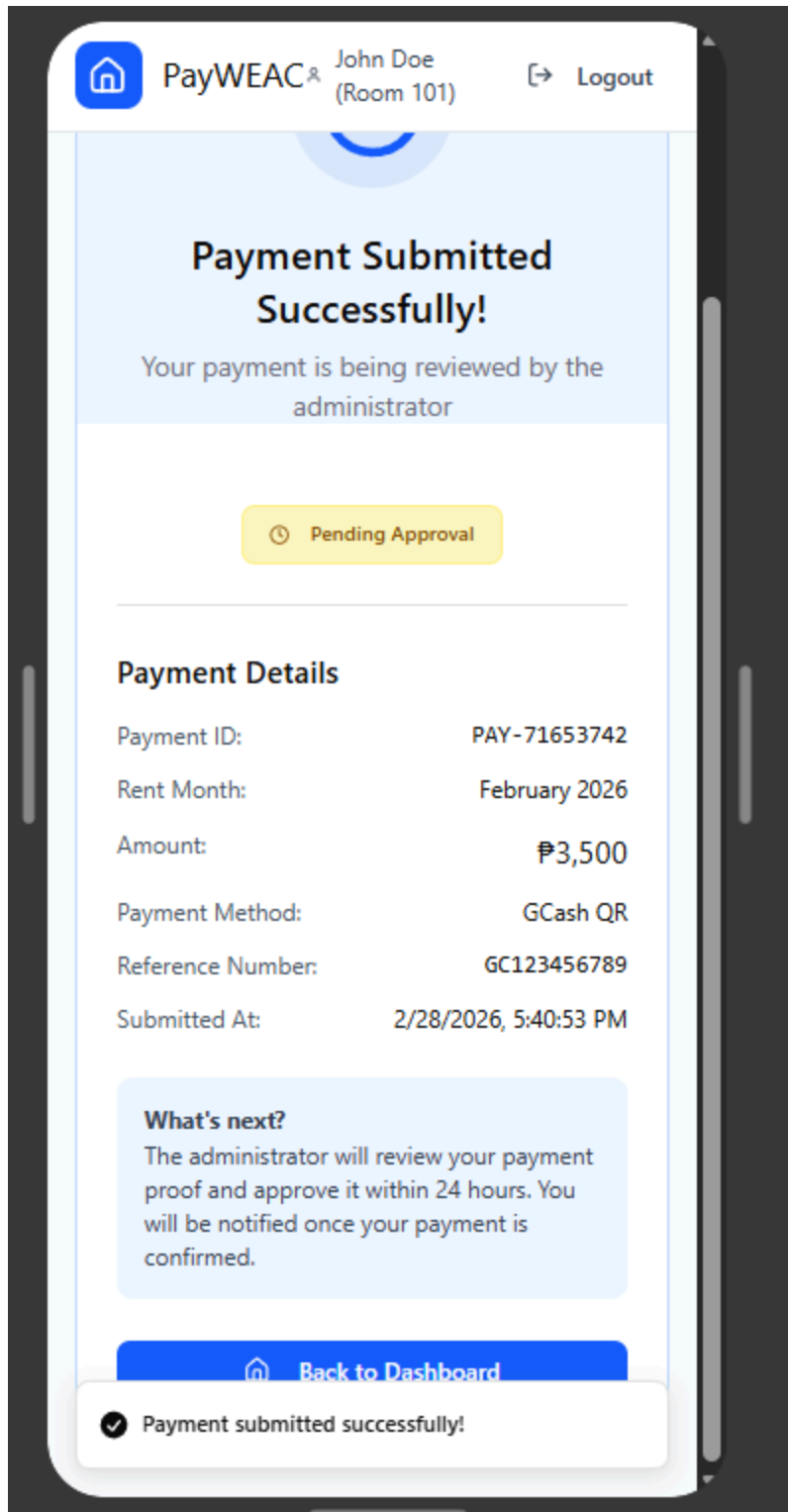
Demo accounts:

Admin: admin@payweac.com

Tenant: tenant@payweac.com







Bottom Navigation

[ Home] [ Search] [ Cart] [ Profile]

Home Screen

Current Rent Status

Due Date Countdown

Pay Now Button

Pay Screen:

- QR Code Display
- Upload Screenshot
- Submit Payment

History Screen:

- List of All Paid and Unpaid Rents
- Payment Status (Approved / Pending / Rejected)

Mobile-Specific Features:

- Touch-friendly buttons (44x44px minimum)
- Simple step-by-step payment flow
- Fast loading rent summary
- Pull-to-refresh for status updates

Design Colors:

Primary: #2563EB

Success: #10B981

Error: #EF4444

Warning: #F59E0B

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

Day 1-2: Project setup and documentation

Day 3-4: Complete FRS and NFR

Day 5-7: System architecture design

Week 2: Detailed Design

Day 1-2: API specification

Day 3-4: Database design

Day 5-6: UI/UX wireframes

Day 7: Implementation plan finalization

Phase 2: Backend Development (Week 3-4)

Week 3: Foundation

Day 1: Spring Boot setup with dependencies

Day 2: Database configuration and entities

Day 3: JWT authentication implementation

Day 4: User registration & login

Day 5: Role-based access (TENANT / ADMIN)

Week 4: Core Features

Day 1: Rent management API

Day 2: Payment submission API

Day 3: Payment approval workflow

Day 4: Error handling & validation

Day 5: API documentation and testing

Phase 3: Web Application (Week 5-6)

Week 5: Frontend Foundation

Day 1: React setup

Day 2: Authentication pages (login, register)

Day 3: Tenant Dashboard

Day 4: Rent listing

Week 6: Complete Web Features

Day 1: Payment submission page

Day 2: Admin dashboard

Day 3: Payment approval interface

Day 4: Responsive design polish

Day 5: API integration and testing

Phase 4: Mobile Application (Week 7-8)

Week 7: Android Foundation

Day 1: Android Studio setup and project structure

Day 2: Authentication screens

Day 3: Rent Status display

Day 4: API service layer

Week 8: Complete Mobile App

Day 1: Payment Submission

Day 2: Status tracking

Day 3: UI polish

Day 4: Testing on emulator/device

Day 5: APK generation and documentation

Phase 5: Integration & Deployment (Week 9-10)

Week 9: Integration Testing

Day 1: End-to-end testing across platforms

Day 2: Bug fixes and optimization

Day 3: Security review

Day 4: Performance testing

Day 5: Documentation updates

Week 10: Deployment

Day 1: Backend deployment (Railway/Render)

Day 2: Web app deployment

Day 3: Mobile APK distribution

Day 4: Final testing

Day 5: Project submission

Milestones:

- **M1 (End Week 2):** All design documents complete
- **M2 (End Week 4):** Backend API fully functional
- **M3 (End Week 6):** Web application complete
- **M4 (End Week 8):** Mobile application complete
- **M5 (End Week 10):** Full system deployed and integrated

Critical Path:

1. Authentication system (Week 3)
2. Product catalog API (Week 3-4))
3. Checkout process (Week 6)
4. Cross-platform testing (Week 9)

Risk Mitigation:

- Start with simplest working version of each feature

- Test integration points early and often
- Keep backup of working versions
- Focus on core functionality before enhancements